

reporte del Laboratorio 9: CI/CD

Repositorio del laboratorio

El código fuente del laboratorio se encuentra disponible en el siguiente repositorio público de GitHub:

<https://github.com/ajperez20/laboratorio-9-nube>

Introducción

En este laboratorio se desarrolló una práctica básica de integración y despliegue continuo utilizando **Jenkins**, **Docker** y una aplicación simple construida con **Node.js** y **Express**. El objetivo principal fue comprender cómo automatizar un flujo CI/CD donde Jenkins clona el código fuente desde GitHub, construye una imagen Docker, ejecuta un contenedor con la aplicación y finalmente limpia los recursos utilizados.

El laboratorio permitió trabajar con conceptos importantes de computación en la nube y DevOps, especialmente la automatización de procesos, el uso de contenedores y la ejecución de pipelines dentro de un entorno controlado. La guía original indicaba levantar Jenkins en Docker, crear una aplicación Node.js, construir su imagen Docker, ejecutar el contenedor y luego configurar un pipeline para automatizar este proceso.

Desarrollo del laboratorio

Primero, se configuró Jenkins dentro de un contenedor Docker. Esto permitió ejecutar Jenkins localmente sin instalarlo directamente en el sistema operativo. Para acceder a Jenkins se utilizó el puerto **8080**, mientras que el puerto **50000** quedó disponible para la comunicación con agentes Jenkins.

Luego se creó una aplicación básica en Node.js utilizando Express. Esta aplicación expone un endpoint principal `/` que responde con un mensaje simple:

```
Hello, World from Docker and Jenkins!
```

Después, se creó un **Dockerfile** para contenerizar la aplicación. Este archivo define la imagen base de Node.js, el directorio de trabajo, la instalación de dependencias, la copia del código fuente y el comando necesario para iniciar la aplicación.

También se organizó el proyecto en una estructura más clara, separando la configuración de Jenkins y el código de la aplicación:

```
LABORATORIO_9-NUBE/
```

```
|
|— docs/
|— jenkins/
|— node-app/
|— .gitignore
```

Esta estructura facilita mantener separados los archivos de infraestructura, documentación y aplicación.

Configuración del pipeline

El punto más importante del laboratorio fue la configuración del pipeline de Jenkins. El pipeline fue diseñado para ejecutar las siguientes etapas:

1. **Checkout:** clonar el repositorio desde GitHub.
2. **Build Docker Image:** construir la imagen Docker de la aplicación.
3. **Remove Previous Container:** eliminar un contenedor anterior si existía.
4. **Run Docker Container:** ejecutar la aplicación dentro de un contenedor Docker.
5. **Smoke Test:** validar que la aplicación estuviera respondiendo correctamente.
6. **Cleanup:** eliminar el contenedor y la imagen creada al finalizar.

Un aprendizaje importante fue que el pipeline original no funcionaba de forma inmediata porque Jenkins se estaba ejecutando dentro de Docker. Para que Jenkins pudiera ejecutar comandos Docker, fue necesario configurar correctamente el acceso al Docker daemon del host. Este fue el principal correctivo técnico del laboratorio.

Además, se mejoró el pipeline para evitar comandos peligrosos como eliminar todos los contenedores del sistema. En lugar de borrar todos los recursos Docker, se configuró la limpieza para eliminar únicamente el contenedor y la imagen asociados al laboratorio.

Validación del resultado

El pipeline fue ejecutado correctamente en Jenkins. En el output de ejecución se pudo verificar que Jenkins realizó todas las etapas esperadas:

- Clonó el repositorio desde GitHub.
- Construyó la imagen Docker de la aplicación.
- Ejecutó el contenedor correctamente.
- Validó la aplicación mediante un smoke test.
- Eliminó los recursos creados al finalizar.
- Terminó con estado exitoso: **Finished: SUCCESS.**

El smoke test fue especialmente importante porque permitió confirmar que la aplicación no solo se construyó y ejecutó, sino que también respondió correctamente a una petición HTTP.

Conocimientos adquiridos

Durante este laboratorio se reforzaron varios conceptos clave:

- Uso de **Docker** para empaquetar aplicaciones.
- Uso de **Jenkins** como servidor de automatización CI/CD.
- Creación de pipelines declarativos con etapas claras.
- Integración entre Jenkins y GitHub.
- Construcción y ejecución de imágenes Docker desde Jenkins.
- Importancia de validar una aplicación con pruebas simples después del despliegue.
- Limpieza controlada de recursos Docker.
- Riesgos de ejecutar Docker desde un contenedor sin una configuración adecuada.

Conclusión

El laboratorio fue completado exitosamente y permitió implementar un flujo básico de CI/CD utilizando Jenkins y Docker. La práctica mostró cómo automatizar el proceso de construcción y ejecución de una aplicación Node.js desde un repositorio de GitHub.

El principal aprendizaje técnico fue comprender que Jenkins, al ejecutarse dentro de Docker, necesita una configuración especial para poder usar comandos Docker. También se identificó la importancia de diseñar pipelines seguros, evitando limpiezas agresivas que puedan afectar otros contenedores del sistema.

En conclusión, este laboratorio permitió aplicar de forma práctica conceptos fundamentales de computación en la nube, automatización, contenedores y DevOps, sentando una base útil para despliegues más avanzados en entornos reales.